

Methylation

Katherine I. Cooper

2026-05-01

Methylation detection

Please run all chunks here and move to the parent file afterwards. This file contains **libraries** and a **function**.

```
## Warning: package 'ggplot2' was built under R version 4.5.3

## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.6
## v forcats    1.0.1      v stringr    1.6.0
## v ggplot2    4.0.3      v tibble     3.3.1
## v lubridate  1.9.5      v tidyr      1.3.2
## v purrr      1.2.1
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
## Loading required package: BiocGenerics
##
## Loading required package: generics
##
##
## Attaching package: 'generics'
##
##
## The following object is masked from 'package:lubridate':
##
##   as.difftime
##
##
## The following object is masked from 'package:dplyr':
##
##   explain
##
##
## The following objects are masked from 'package:base':
##
##   as.difftime, as.factor, as.ordered, intersect, is.element, setdiff,
##   setequal, union
##
##
```

```

## Attaching package: 'BiocGenerics'
##
##
## The following object is masked from 'package:dplyr':
##
##   combine
##
##
## The following objects are masked from 'package:stats':
##
##   IQR, mad, sd, var, xtabs
##
##
## The following objects are masked from 'package:base':
##
##   anyDuplicated, aperm, append, as.data.frame, basename, cbind,
##   colnames, dirname, do.call, duplicated, eval, evalq, Filter, Find,
##   get, grep, grepl, is.unsorted, lapply, Map, mapply, match, mget,
##   order, paste, pmax, pmax.int, pmin, pmin.int, Position, rank,
##   rbind, Reduce, rownames, sapply, saveRDS, table, tapply, unique,
##   unsplit, which.max, which.min
##
##
## Loading required package: S4Vectors
##
## Loading required package: stats4
##
##
## Attaching package: 'S4Vectors'
##
##
## The following objects are masked from 'package:lubridate':
##
##   second, second<-
##
##
## The following objects are masked from 'package:dplyr':
##
##   first, rename
##
##
## The following object is masked from 'package:tidyr':
##
##   expand
##
##
## The following object is masked from 'package:utils':
##
##   findMatches
##
##
## The following objects are masked from 'package:base':
##
##   expand.grid, I, unname

```

```

##
##
## Loading required package: IRanges
##
##
## Attaching package: 'IRanges'
##
##
## The following object is masked from 'package:lubridate':
##
##   %within%
##
##
## The following objects are masked from 'package:dplyr':
##
##   collapse, desc, slice
##
##
## The following object is masked from 'package:purrr':
##
##   reduce
##
##
## The following object is masked from 'package:grDevices':
##
##   windows
##
##
## Loading required package: Seqinfo
##
## Loading required package: GenomicRanges
##
## Loading required package: AnnotationDbi
##
## Loading required package: Biobase
##
## Welcome to Bioconductor
##
##   Vignettes contain introductory material; view with
##   'browseVignettes()'. To cite Bioconductor, see
##   'citation("Biobase")', and for packages 'citation("pkgname")'.
##
##
##
## Attaching package: 'AnnotationDbi'
##
##
##
## The following object is masked from 'package:dplyr':
##
##   select
##
##
##
##

```

```
##
##
## Loading required package: GenomeInfoDb
##
## Loading required package: BSgenome
##
## Loading required package: Biostrings
##
## Loading required package: XVector
##
##
## Attaching package: 'XVector'
##
##
## The following object is masked from 'package:purrr':
##
##     compact
##
##
## Attaching package: 'Biostrings'
##
##
## The following object is masked from 'package:base':
##
##     strsplit
##
## Loading required package: BiocIO
##
## Loading required package: rtracklayer
```

CG dinucleotide counter:

The following code cell contains a function that automatically compiles CG dinucleotides from a reference genome. To view the code, uncollapse the cell; however, this is non-essential for running the function.

To load the function, please click the green arrow on the right-side of the code cell. This will load the function into your environment. It is called: countCpG.

```
countCpG <- function(df, GeneIDs, flankRegion, species){

  # Set-up for the species specific portion

  #Which packages are needed
  message(paste("Loading packages for", species, "."))

  Species.pkgs <- switch(species,
    "Hs" = c("BSgenome.Hsapiens.UCSC.hg38",
              "org.Hs.eg.db"),
    "Mm" = c("BSgenome.Mmusculus.UCSC.mm39",
              "org.Mm.eg.db"),
    "Gg" = c("BSgenome.Ggallus.UCSC.galGal5",
              "org.Gg.eg.db"),
```

```

    stop("Invalid species choice. Supported options: Homo sapiens (Hs), Mus musculus (Mm), Gallus gallus
  )

sapply(Species.pkgs, library, character.only = T)

#Assigning the ensembl

ensembl <- switch(species,
  "Hs" = useMart("ensembl",
    dataset = "hsapiens_gene_ensembl",
    host = "https://may2025.archive.ensembl.org"),
  "Mm" = useMart("ensembl",
    dataset = "mmusculus_gene_ensembl",
    host = "https://dec2021.archive.ensembl.org"),
  "Gg" = useMart("ensembl",
    dataset = "ggallus_gene_ensembl",
    host = "https://dec2021.archive.ensembl.org"),
  stop("Invalid species choice. Supported options: Homo sapiens (Hs), Mus musculus (Mm), Gallus gal

BSgenome.sp <- switch(species,
  "Hs" = BSgenome.Hsapiens.UCSC.hg38,
  "Mm" = BSgenome.Mmusculus.UCSC.mm39,
  "Gg" = BSgenome.Ggallus.UCSC.galGal5,
  stop("Unknown BSgenome object")
)

genes = df[[GeneIDs]]

#Mapping the ensembl names to the symbols

#Getting which ensembl db

pkg.map <- c(
  "Mm" = "org.Mm.eg.db",
  "Hs" = "org.Hs.eg.db",
  "Gg" = "org.Gg.eg.db"
)

org.db <- get(pkg.map[[species]])

Ensembl_names <- mapIds(org.db,
  keys = genes, keytype = "SYMBOL", column = "ENSEMBL")

##loop set-ups

#Empty vector for the coords
coords <- list()

#Flanking regions
flanks <- flankRegion

#Counter for final df
counter = 1

```

```

##Loop 1: Assigning TSS coordinate
for (n in genes) {

  #Message for which gene

  message(paste0("(", counter, "/", length(Ensembl_names), ") Processing: ", n))

  #Checking for NA

  if(is.na(Ensembl_names[n]) || is.null(Ensembl_names[n])) {
    message(paste("Unknown gene SYMBOL ID for", n, "Skipping gene."))
    next
  }

  #Assinging the emsembl for the loop
  Ensembl_id <- Ensembl_names[n]

  #Getting the TSS coordinate
  tss_info <- getBM(
    filters = "ensembl_gene_id",
    values = Ensembl_id,
    attributes = c("chromosome_name", "transcription_start_site", "strand"),
    mart = ensembl
  )

  #Skipping no TSS
  if(is.null(tss_info) || nrow(tss_info) == 0) {
    message(paste("Skipping gene:", n, "-No TSS information found in BioMart"))
    next
  }

  #Assigning the TSS

  tss_info <- tss_info[1, ]

  ##Loop 2: Finding CGs in the gene per each flanking regions
  for (i in flanks) {

    #Adding/subtracting flanks
    start_coord <- tss_info$transcription_start_site - i
    end_coord <- tss_info$transcription_start_site + i

    #Ensuring the start/stop are not flipped because of - strand
    if (start_coord > end_coord) {
      temp <- start_coord
      start_coord <- end_coord
      end_coord <- temp
    }

    #Pulling the DNA sequence info given the start/stop region

```

```

tss_range <- GRanges(
  seqnames = paste0("chr", tss_info$chromosome_name),
  ranges = IRanges(start = start_coord, end = end_coord),
  strand = ifelse(tss_info$strand == 1, "+", "-")
)

#Getting the actual sequence
tss_seequence <- getSeq(BSgenome.sp, tss_range)

#Counting CG dinucleotides
CpG_count <- str_count(as.character(tss_seequence), "CG")

#Data frame accumulation
coords[[counter]] <- data.frame(
  Gene_id = n,
  CpGs = CpG_count,
  flanking_region = as.character(i)
)

#Counter
counter <- counter + 1

#Message for done
message(paste("Finished processing", n, "."))

}

}
cpgCounts <- do.call(rbind, coords)

#Final message
message("---- Processing complete!----")

return(cpgCounts)
}

```

Function documentation: The following contains an example using a manually input data frame, `genesTest`. It is essential to have the proper syntax of the genes' symbols for mapping purpose: ex. `RHO` is incorrect in this instance. `Rho` is correct.

There are three arguments the user needs to input:

- `df`: Identify the data containing the gene symbols
- `GeneIDs`: Which column contains the gene symbol IDs in the specified `df`. This needs to be in quotation marks.
- `flankRegion`: Specify the buffer regions around the TSS. This can be manually input or a vector.

Common errors: start_coord length is zero implies an improper gene name. I am working on making a more human-interpretable error code.

```
sessionInfo()
```

```
## R version 4.5.2 (2025-10-31 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26200)
##
## Matrix products: default
##   LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=English_United States.utf8
## [2] LC_CTYPE=English_United States.utf8
## [3] LC_MONETARY=English_United States.utf8
## [4] LC_NUMERIC=C
## [5] LC_TIME=English_United States.utf8
##
## time zone: America/New_York
## tzcode source: internal
##
## attached base packages:
## [1] stats4      stats      graphics  grDevices  utils      datasets  methods
## [8] base
##
## other attached packages:
## [1] TxDb.Mmusculus.UCSC.mm39.knownGene_3.22.0
## [2] BSgenome.Mmusculus.UCSC.mm39_1.4.3
## [3] BSgenome.Hsapiens.UCSC.hg38_1.4.5
## [4] BSgenome_1.78.0
## [5] rtracklayer_1.70.1
## [6] BiocIO_1.20.0
## [7] Biostrings_2.78.0
## [8] XVector_0.50.0
## [9] GenomeInfoDb_1.46.2
## [10] org.Mm.eg.db_3.22.0
## [11] org.Hs.eg.db_3.22.0
## [12] biomaRt_2.66.1
## [13] GenomicFeatures_1.62.0
## [14] AnnotationDbi_1.72.0
## [15] Biobase_2.70.0
## [16] GenomicRanges_1.62.1
## [17] Seqinfo_1.0.0
## [18] IRanges_2.44.0
## [19] S4Vectors_0.48.0
## [20] BiocGenerics_0.56.0
## [21] generics_0.1.4
## [22] lubridate_1.9.5
## [23] forcats_1.0.1
## [24] stringr_1.6.0
## [25] dplyr_1.1.4
## [26] purrr_1.2.1
## [27] readr_2.1.6
```



```

## [28] tidyr_1.3.2
## [29] tibble_3.3.1
## [30] ggplot2_4.0.3
## [31] tidyverse_2.0.0
##
## loaded via a namespace (and not attached):
## [1] DBI_1.3.0 bitops_1.0-9
## [3] httr2_1.2.2 rlang_1.1.7
## [5] magrittr_2.0.4 otel_0.2.0
## [7] matrixStats_1.5.0 compiler_4.5.0
## [9] RSQLite_2.4.6 png_0.1-8
## [11] vctrs_0.7.1 pkgconfig_2.0.3
## [13] crayon_1.5.3 fastmap_1.2.0
## [15] dbplyr_2.5.2 Rsamtools_2.26.0
## [17] rmarkdown_2.31 tzdb_0.5.0
## [19] UCSC.utils_1.6.1 tinytex_0.59
## [21] bit_4.6.0 xfun_0.56
## [23] cachem_1.1.0 cigarillo_1.0.0
## [25] jsonlite_2.0.0 progress_1.2.3
## [27] blob_1.3.0 DelayedArray_0.36.0
## [29] BiocParallel_1.44.0 parallel_4.5.0
## [31] prettyunits_1.2.0 R6_2.6.1
## [33] stringi_1.8.7 RColorBrewer_1.1-3
## [35] SummarizedExperiment_1.40.0 knitr_1.51
## [37] Matrix_1.7-3 timechange_0.4.0
## [39] tidyselect_1.2.1 rstudioapi_0.18.0
## [41] dichromat_2.0-0.1 abind_1.4-8
## [43] yaml_2.3.12 codetools_0.2-20
## [45] curl_7.0.0 lattice_0.22-7
## [47] withr_3.0.2 KEGGREST_1.50.0
## [49] S7_0.2.1 evaluate_1.0.5
## [51] BiocFileCache_3.0.0 pillar_1.11.1
## [53] BiocManager_1.30.27 filelock_1.0.3
## [55] MatrixGenerics_1.22.0 RCurl_1.98-1.17
## [57] hms_1.1.4 scales_1.4.0
## [59] glue_1.8.0 tools_4.5.0
## [61] GenomicAlignments_1.46.0 XML_3.99-0.22
## [63] grid_4.5.0 restfulr_0.0.16
## [65] cli_3.6.5 rappdirs_0.3.4
## [67] S4Arrays_1.10.1 gtable_0.3.6
## [69] digest_0.6.39 SparseArray_1.10.8
## [71] rjson_0.2.23 farver_2.1.2
## [73] memoise_2.0.1 htmltools_0.5.9
## [75] lifecycle_1.0.5 httr_1.4.8
## [77] bit64_4.6.0-1

```